

How Pradarshak AI Actually Works

A plain-English walkthrough of the entire system — from user's first message to the AI's response — with just enough code to understand what's happening under the hood.

 September
2026

 Backend: Express +
TypeScript

 AI: Gemini 2.5
Flash

 DB: PostgreSQL +
PostGIS

1

What is Pradarshak AI?

The product in one sentence, and the problem it solves.

The Indian government — specifically NSFDC (National Scheduled Castes Finance and Development Corporation) — offers subsidised loans to Scheduled Caste beneficiaries. These are people from economically weaker SC families (annual income below ₹5 lakh) who want to start a business, pursue higher education, or buy equipment. The government offers them loans at very low interest rates: 6.5% to 8%, which is far cheaper than a bank would offer.

The problem is that this system is confusing. There are multiple different loan schemes, each with different loan limits, rules, and interest rates. The money doesn't come directly from the government — it goes through 100+ "Channel Partners" (state government agencies, rural banks, NBFCs). A person sitting in a village has no way to know: which scheme fits me? How much will I pay each month? Who near me can actually process this?

Pradarshak AI solves exactly this. A person types their situation in plain language — in Hindi, Marathi, or English — and the system tells them which scheme fits them, calculates their exact EMI, and shows them the nearest authorized partner to visit.

The key design decision

The AI's job is only to *understand and explain*. All actual numbers — interest rates, loan limits, EMI figures, partner addresses — come from the database or real math formulas. The AI never makes up a number.

2

The System at a Glance

Four pieces working together.

Piece	What it is	What it does
Frontend	Next.js website (chat UI)	The screen the user sees and types into
Backend	Express.js server (Node.js)	Receives messages, runs logic, talks to AI and DB
Database	PostgreSQL on NeonDB (cloud)	Stores loan scheme data, partner locations, chat history
AI	Gemini 2.5 Flash via OpenRouter	Understands user intent, writes natural language responses

The backend runs on **port 4000** on the server. The frontend (Next.js) runs on **port 3000**. There's also a separate admin panel on port 3001 — a simple management interface for the team to manage schemes and partners.

The database is not hosted locally. It lives on **NeonDB**, which is a cloud service that runs PostgreSQL for you. The backend connects to it over the internet using a connection string — think of it like a URL that includes the username, password, and database name. SSL is required (so the connection is encrypted).

The AI is accessed through **OpenRouter**, which is essentially a gateway that sits in front of multiple AI providers. Instead of integrating directly with Google's API, we call OpenRouter, tell it which model we want (Gemini 2.5 Flash), and it handles the rest. This means switching models in the future requires changing one line in the config, not rewriting the integration.

3

What Happens When You Send a Message

The full journey from "I want to open a shop" to the AI's reply — step by step.



User types a message in the chat UI

The Next.js frontend sends an HTTP POST request to the backend at `/api/chat`. The request body contains the user's message text and a chat ID (to identify which conversation this belongs to).



Backend checks if the user is logged in

The backend looks for a JWT token in the request's Authorization header. If found and valid, it knows who the user is. If there's no token, that's fine too — the user can still chat, their history just won't be saved to the database. Both logged-in and guest users work.



If logged in: save the message to the database

Before processing anything, the user's message is inserted into the `chat_messages` table in PostgreSQL. This is how chat history gets saved — so when you come back tomorrow, your conversation is still there.



The message goes to the ChatOrchestrator

This is the brain of the system. It's a TypeScript file that manages the entire conversation and decides how to respond. Everything interesting happens here.



Response comes back and gets saved

Once the AI's response is ready, it gets saved to the database (for logged-in users) and sent back to the frontend, which shows it in the chat.

4

The Brain: ChatOrchestrator

This is the most important part of the entire system. Understanding this means understanding everything.

Step 1 — Recall the conversation

Every conversation is stored in memory on the server as a "session" object. When your message arrives, the orchestrator looks up the session for your chat ID. It finds the full history of what was said before — every user message and every assistant reply.

This is important because the AI doesn't have memory by itself. We have to manually pass the entire conversation history to it every single time. The session object also remembers what language you've been writing in and the results of previous tool calls (more on tools in a moment).

Important to know about sessions

Sessions are stored in server RAM, not the database. If the backend server restarts, all active sessions are lost. However, for logged-in users, the chat history is in the database, so the UI can still show past messages. Sessions also auto-delete after 1 hour of inactivity.

Step 2 — Detect the language

Before calling the AI, the system quickly detects whether the user is writing in English, Hindi, or Marathi. This is done with a simple counting trick — no AI involved.

The code counts how many Devanagari script characters (the script used for Hindi and Marathi) are in the message. If more than 10% of the characters are Devanagari, it's not English. Then it checks for specific grammar words — Hindi uses "मुझे, चाहिए, है" while Marathi

uses "मला, हवे, आहे". Whichever language has more matching words, that's the detected language.

This detection is used to tell the AI which language to respond in. If the user writes in Hindi, the system prompt tells Gemini to reply in Hindi.

Step 3 — Build the message package for the AI

The orchestrator assembles everything the AI needs to know into a list of messages:

WHAT GETS SENT TO GEMINI EVERY TURN

1. **SYSTEM:** Instructions on how to behave (never invent numbers, reply in Hindi, etc.)
2. **SYSTEM:** "Known context: here are the real scheme results from 2 messages ago ... "
3. **USER:** "I want to open a tailoring shop, income is 3 lakh"
4. **ASSISTANT:** "Great! What area are you in?"
5. **USER:** "I am in Nagpur"
 ↑ current message goes here

The "Known context" injection (item 2) is clever — when the AI previously fetched scheme data, the orchestrator saved a compact snapshot of that real data and injects it back every turn. This way, the AI can reference "the scheme we discussed earlier" without needing to call the database again.

Step 4 — The Tool-Calling Loop (the core mechanic)

This is where it gets interesting. We send the full conversation to Gemini, but we also send it a list of "tools" — functions it's allowed to call. Each tool has a name, a description of when to use it, and the parameters it needs.

Gemini then makes a decision:

- **Option A — Ask a question:** "I don't have enough information. I need to ask the user which city they're in." → Gemini replies with a question, no tool called.
- **Option B — Call a tool:** "I have enough info — purpose is tailoring, income is 3 lakh. I'll call recommend_schemes." → Gemini returns a tool_call instead of text.
- **Option C — Answer directly:** For general questions about the application process or NSFDC background, it can answer from its own knowledge.

WHAT GEMINI RETURNS WHEN IT WANTS TO CALL A TOOL (SIMPLIFIED)

```
tool_calls: [{  
  name: "recommend_schemes",  
  arguments: {  
    purpose: "tailoring shop",  
    family_income_rs: 300000,  
    location: "Nagpur"  
  }  
}]
```

When the orchestrator sees a tool_call in the response, it runs the corresponding function in our backend code. The result (real data from the database) is then fed back to Gemini. Gemini reads the real data and writes a plain-language explanation.

This loop can run up to 3 times in a single turn, in case the AI needs to call multiple tools in sequence. In practice, most turns complete in 2 rounds: one tool call + one explanation.

Why this architecture matters

The AI never sees the database directly. Our backend code fetches real data, then shows it to the AI and says "here's the real information, now explain it to the user." This is how we guarantee the AI never invents loan amounts or interest rates.

5

The Five Tools

What the AI can actually do — and what runs under the hood for each one.

Tool 1: recommend_schemes

The AI calls this when it has enough information to suggest loan schemes. It passes what it knows: what the loan is for, how much income the family has, whether it's for education, the applicant's gender, etc.

Under the hood, this fetches all active schemes from the database, runs a scoring algorithm across each one (explained in detail in Section 6), and returns the top 3 matches. If a city was mentioned, it also finds the nearest Channel Partner for that area in the same call.

Tool 2: calculate_emi

The AI calls this when the user wants to know their monthly payment. The AI passes: loan amount, interest rate (from the scheme it already recommended), tenure in months, and moratorium period.

The EMI (Equated Monthly Instalment) is calculated with the standard banking formula — the same one every bank uses. The result includes the monthly EMI, the total interest paid over the entire loan period, and the total amount paid back. The AI then just reads these numbers and tells the user.

THE EMI FORMULA (THIS RUNS IN OUR CODE, NOT IN AI)

$$EMI = P \times r \times (1+r)^n / ((1+r)^n - 1)$$

P = principal (loan amount)

r = monthly interest rate (annual rate ÷ 12 ÷ 100)

n = number of months

Example: ₹5 lakh at 7% for 5 years (60 months)

$$r = 7/100/12 = 0.00583$$

$$EMI = ₹9,900/month$$

What is a moratorium? It's a grace period at the start of the loan where you don't pay anything — typically used for education loans (you get a few months after completing your course before payments start). During this period, interest still accumulates on the loan. Our code accounts for this by computing the inflated principal after the moratorium period before applying the EMI formula.

Tool 3: find_partners

Finds the nearest authorized Channel Partners for a given city. Uses geo-spatial database queries (explained in Section 7). Returns partner name, address, phone, and distance in km.

Tool 4: get_required_documents

Returns the standard list of documents needed to apply. If a specific scheme is known, it tailors the list slightly (e.g., education loans need an admission letter; business loans need a project report). The document list itself is hardcoded — it doesn't vary enough to need a database lookup.

Tool 5: compare_schemes

Fetches two or more named schemes from the database side by side, so the AI can compare them. If the user doesn't name specific schemes, it defaults to comparing the top two active schemes.



How Schemes Are Matched — The Scoring System

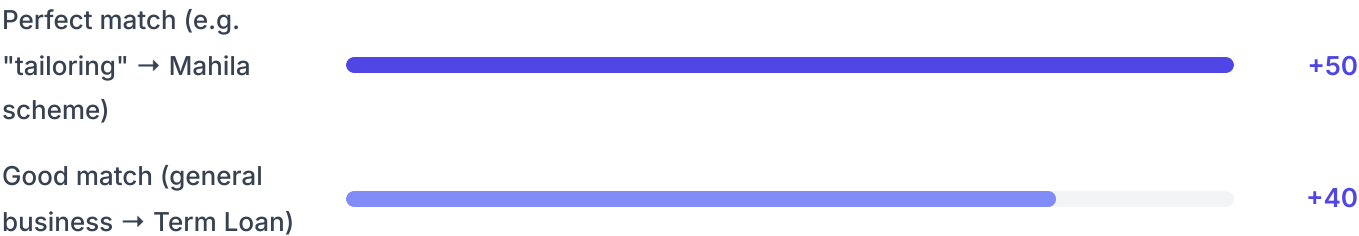
Every scheme gets a numeric score. The top 3 are shown. Here's how the score is calculated.

When `recommend_schemes` runs, it fetches every active scheme from the database. For each scheme, it calculates a score based on how well it matches what the user told us. Schemes with a very low score (below -30) are dropped entirely. The rest are sorted from highest to lowest score, and the top 3 are returned to the AI.

The score has five components:

1. Purpose Match — Does the loan purpose fit this scheme?

The code has lists of keywords grouped by category. If the user said "tailoring shop", it matches the *business words* category. If they said "electric rickshaw", it matches *green/clean energy*. The scheme database has information on what types of businesses or purposes each scheme covers.



Weak or partial match		+15
Wrong category (farming → business loan)		-30

2. Income Eligibility — Is the family income within the scheme's limits?

Each scheme has a maximum allowed family income (most are ₹5 lakh/year). If the user's income exceeds the scheme's limit, the scheme gets -10 points. If it's within limits, it gets +20. If we don't know the income, it still gets a neutral +10 (benefit of the doubt).

3. Loan Amount Fit — Does the requested amount fit within the scheme cap?

If the user asks for ₹10 lakh but the scheme only goes up to ₹1.4 lakh (Micro Finance Scheme), that's a strong mismatch (-20). If the amount fits, it gets +25.

4. Education vs Business — Strong filter

This is the most aggressive filter. If the user is asking for an education loan and we're scoring a business loan scheme, that scheme gets -150. This effectively eliminates it from consideration. The reverse is also true. This ensures education and business loans never get mixed up.

5. Gender Eligibility — Women-only schemes

Some schemes like *Mahila Samriddhi Yojana* are exclusively for women. If a male applicant is matched against this scheme, it gets a score of -300 — completely eliminating it. If a female applicant matches it, she gets +25 instead. This ensures women-only schemes only appear for women.

Why use a scoring system instead of just filtering?

Hard filters are binary — either you qualify or you don't. A scoring system allows *ranking*. Two schemes might both technically qualify, but one is a much better fit. Scores let us surface the most relevant one at the top, which is what the user actually needs.

How Partner Location Search Works

Finding the nearest authorized Channel Partner using geo-spatial database queries.

Step 1 — Convert city name to coordinates

The user says "I am in Nagpur." Our code maintains a pre-built lookup table of 60+ Indian cities with their GPS coordinates (latitude and longitude). Nagpur maps to [21.1458, 79.0882].

If the city isn't in the table (e.g., a smaller town), the code makes a call to **OpenStreetMap's Nominatim API** — a free geocoding service. You give it a city name, it gives back coordinates. No cost, no API key needed.

Step 2 — Query the database with those coordinates

The partners table in the database has a special column called `location` that stores GPS coordinates for each partner branch. This uses a PostgreSQL extension called **PostGIS**, which adds geographic intelligence to the database.

THE QUERY THAT FINDS NEARBY PARTNERS (SIMPLIFIED TO EXPLAIN)

```
SELECT name, city, phone, distance_km
FROM partners
WHERE
  is_active = TRUE
  AND within_150km_of(Nagpur coordinates)
  AND can_process_this_scheme_category
ORDER BY distance_km ASC
LIMIT 4
```

PostGIS computes the distance between the user's city and each partner's stored location using real geographic math (it accounts for the curvature of the Earth). The radius is 150 km. If no partners are found within 150 km, it runs a second query with no distance limit, returning the closest partners in the entire country as a fallback.

The partners table also has `eligible_categories` (a list of which types of schemes this partner can process) and `is_active` (whether the partner is currently operational). The query filters by both.

How Conversation Memory Works

Two layers of memory — one in RAM, one in the database.

Layer 1: In-memory sessions (for the active conversation)

While you're actively chatting, your entire conversation history is stored in the server's RAM as a JavaScript object. This is fast — no database call needed to look up your history. Every time you send a message, the orchestrator retrieves this object, adds your new message to it, builds the full message package for the AI, and saves the AI's response back into it.

The session also stores a "last context" snapshot — a compact version of the most recent tool result. This lets the AI reference "the scheme we found earlier" in follow-up messages without having to query the database again.

Layer 2: Database persistence (for long-term history)

For logged-in users, every message (yours and the AI's) is written to the chat_messages table in PostgreSQL. This is the permanent record. Even if the server restarts and the in-memory session is gone, the chat history can be loaded from the database when the user comes back.

Layer	Where	How long	Who gets it
In-memory session	Server RAM	1 hour (auto-deleted)	Everyone (guests + users)
Database messages	PostgreSQL (NeonDB)	Forever	Logged-in users only

The System Prompt — Instructions for the AI

The most important thing we tell Gemini before every conversation.

Every single API call to Gemini starts with a system prompt — a block of text that tells the AI how to behave. Think of it as a job description. Here are the most important rules we give it:

Rule	Why it matters
Never invent interest rates, loan limits, or EMI figures	These are financial decisions — wrong numbers cause real harm
If you need a number, call the matching tool — don't guess	Forces the AI to fetch real data, not hallucinate
If you're missing info, ask ONE question at a time	Prevents overwhelming the user with a form-like list of questions
Respond in the same language the user is writing in	Automatic multilingual support without separate translated UIs
Never use markdown formatting (no bold, no bullet lists)	The chat UI renders plain text only — formatting symbols would show as ugly characters
Be warm and conversational, not robotic	The target users may not be tech-savvy — tone matters

There's also a post-processing step after every AI response: a function called `stripMarkdown` runs on the AI's text and removes any formatting symbols in case Gemini didn't follow the formatting rule perfectly. It strips `#` heading symbols, `**bold**` markers, bullet point dashes, and table pipes. This is a safety net.

Response Types and Quick Actions

The AI's response is more than just text — it includes metadata the frontend uses.

Every response from the backend includes a `type` field that tells the frontend what kind of response this is. The frontend uses this to decide how to display things — for example, scheme

responses might show a card with scheme details, EMI responses might show a summary box.

Type	When it's used	Quick actions shown
schemes	After scheme recommendations	Calculate EMI, Documents checklist, Find partner, Compare schemes
emi	After EMI calculation	Find partner, Documents checklist
partners	After partner search	Documents needed, Calculate EMI
documents	After document list	Find nearest partner, Calculate EMI
text	General answers, greetings	Explore schemes, Education loan, Calculate EMI, Find partner
question	When AI asks a clarifying question	None — waiting for user to answer

Quick Actions are pre-written button labels that appear below every AI message. Clicking one sends a pre-set message — like tapping "Calculate EMI" sends the text "Calculate the EMI for this". It's a shortcut so users don't have to type common follow-up questions.

Responses for scheme recommendations also include a disclaimer — a note in the detected language reminding the user that final eligibility is subject to document verification by the Channel Partner. This is legally important for a financial product.

11

A Complete Example — Start to Finish

Walking through every single step for one real conversation turn.

User types: "मुझे किराना दुकान के लिए लोन चाहिए, परिवार की आय 2.5 लाख है, मैं नागपुर में हूँ"

(Translation: "I need a loan for a grocery shop, family income is 2.5 lakh, I am in Nagpur")

1

Language detection → Hindi detected

Devanagari characters found. "मुझे" and "चाहिए" are Hindi markers. Session language set to hi.

2

System prompt is built in Hindi mode

The prompt tells Gemini: "Reply in Hindi (Devanagari script). Never invent numbers. Use the tools."

3

Round 1 — Gemini decides to call a tool

It has: purpose = "grocery shop", income = ₹2.5L, location = "Nagpur". It calls `recommend_schemes({ purpose: "grocery shop", family_income_rs: 250000, location: "Nagpur" })`

4

SchemeEngine runs scoring on all schemes

"Grocery shop" matches business words → Term Loan gets +40, Mahila scheme gets -15 (business, not women-specific), Kisan scheme gets -30 (agriculture only). Income ₹2.5L is within all limits → +20 each. Term Loan ends up highest scored. Top 3 returned.

5

Simultaneously: nearest partner found via PostGIS

"Nagpur" → coordinates [21.1458, 79.0882] → PostGIS query finds 3 active partners within 150km → closest returned.

6

Round 2 — Gemini reads real data and writes the reply in Hindi

The scheme data (name, rates, limits, moratorium) and nearest partner info are fed to Gemini. It writes a warm, plain-language response in Hindi explaining the best scheme, the interest rate, and which partner to visit — without inventing a single number.

7

Response is saved and returned

Markdown is stripped as safety net. Response type is set to schemes. Quick actions are attached (EMI, Documents, Partner). Hindi disclaimer is added. Everything saved to DB. Frontend shows the reply.

What Lives in the Database

Five tables, each with a clear job.

Table	What it stores	Key columns
schemes	All NSFDC loan schemes — this is the core data	name, category, max_loan_lakh, interest_rate_min/max, moratorium, eligible_project_types, gender_eligibility, active
partners	All Channel Partner branches with their GPS location	name, partner_type, city, phone, location (GPS), eligible_categories, npa_percent, is_active
chats	A record for each conversation thread	id, user_id, title, created_at, updated_at
chat_messages	Every individual message in every chat	chat_id, role (user/assistant), content, type, data (JSON), quick_actions (JSON)
users	Registered user accounts	id, email, password (hashed), created_at

The `partners.location` column is a special PostGIS geography type — not just text like "Nagpur". It stores actual GPS coordinates that the database engine can do distance math on directly.

The `chat_messages.data` column stores JSON — the raw tool result (scheme objects, EMI numbers, partner list). The frontend uses this to render rich cards and summaries, not just plain text.



The One Paragraph Summary

When you type a message, it hits an Express.js server. The server checks if you're logged in, saves your message to PostgreSQL, then hands control to the **ChatOrchestrator**. The Orchestrator loads your conversation history from memory, detects your language, and sends everything to **Gemini 2.5 Flash** via OpenRouter — along with a set of tools Gemini is allowed to call. Gemini reads the conversation, decides it needs to fetch real scheme data, and returns a tool call. The backend runs the **SchemeEngine** — a scoring algorithm that pulls all schemes from the database and ranks them based on purpose, income, loan amount, education status, and gender. If a location was mentioned, a **PostGIS geo-spatial query** finds the nearest authorized Channel Partner. All this real data goes back to Gemini, which writes a natural-language response in the user's language — without inventing a single number. The response is saved to the database, bundled with quick-action buttons and a disclaimer, and sent back to the chat UI.